

ATELIER SGBD –PL/SQL

Hmida HMIDA

PLAN

2

- Éléments de base du PL/SQL
- Les curseurs
- Procédures et fonctions
- Les exceptions
- Les déclencheurs
- Administration des utilisateurs

1 Éléments de base de PL/SQL

Introduction à PL/SQL(1)

4

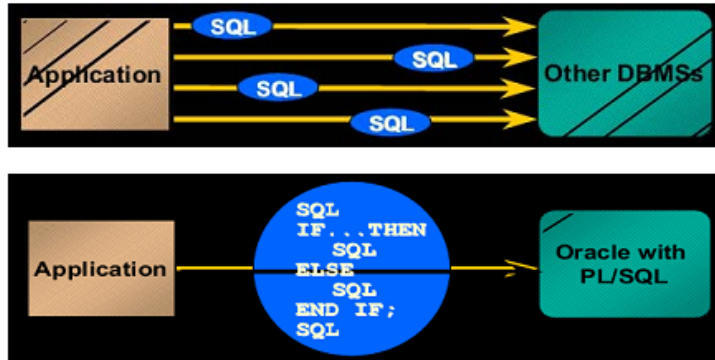
- PL : Procedural Language
- C'est une extension à SQL
 - ▣ LMD et Interrogation SQL y sont inclus
 - ▣ Fonctionnalités des langages de programmation
- Pourquoi
 - ▣ SQL est non procédural
 - ▣ Lier plusieurs requêtes avec des variables
 - ▣ Traitements complexes incluant des itérations, des tests

Introduction à PL/SQL(2)

5

□ Avantages

▣ performance



Introduction à PL/SQL(3)

6

□ Avantages

▣ Gestion des erreurs

▣ Structurer le développement

▣ Portabilité et intégration à d'autres langages de programmation

Blocs de code(1)

7

□ Types

▣ Anonyme

■ Procédure anonyme

▣ Procédures et fonctions

■ Blocs nommés et paramétrés, puis invoqués

■ Triggers invoqués implicitement

□ Un bloc peut contenir d'autres blocs

Blocs de code(2)

8

□ Bloc anonyme

DECLARE -- optionnel

Variables, curseurs, exceptions définies

BEGIN --obligatoire

Instructions SQL & PL/SQL

EXCEPTION --optionnel

Action à réaliser quand une exception est levée

END;-- obligatoire

Blocs de code(3)

9

- Begin et End sont obligatoires
- ; à la fin de chaque instruction et après End
- Les commentaires
 - ▣ Ligne unique : --
 - ▣ Plusieurs lignes : /* ... */

Variables et types(1)

10

- Déclaration
 - identificateur [CONSTANT] type [NOT NULL] [:= | DEFAULT expr];
- ▣ Identificateur : nom de la variable
 - 30 caractères au plus
 - Commence par une lettre
 - Peut contenir lettres, chiffres, _, \$ et #
 - Pas sensible à la casse
 - Une variable déclarée par ligne
- Portée par bloc

Variables et types(2)

11

- Remarques
 - ▣ Adopter une convention pour nommer les variables
 - ▣ Initialiser les variables déclarées NOT NULL
 - ▣ Les noms des variables doivent être différents des noms des colonnes de tables
 - v_empno (variable), g_deptno (globale), c_emp (CURSOR)

Variables et types(3)

12

- Types
 - ▣ Scalaires : types des colonnes
 - ▣ Composés : enregistrement
 - ▣ Références : pointeur
 - ▣ LOB : emplacement pour Large Objects(images)
 - ▣ Collections
 - ▣ Objet

Variables et types(4)

13

- Types scalaires (norme SQL et Oracle)
 - ▣ VARCHAR2 [(maximum_length)]
 - ▣ NUMBER[(precision,scale)]
 - ▣ DATE
 - ▣ CHAR[(maximum_length)]
 - ▣ LONG : 2 147 483 647 octets
 - ▣ BOOLEAN : True, False ou NULL
 - ▣ INTEGER

Variables et types(5)

14

□ Exemples

■ c	CHAR(1);
■ name	VARCHAR2(10) := 'Scott';
■ total	NUMBER(9, 2) := 0;
■ order	DATE := SYSDATE + 7;
■ Ship	DATE;
■ pi	CONSTANT NUMBER (3, 2) := 3.14;
■ done	BOOLEAN NOT NULL := TRUE;
■ V_date	DATE := TO_DATE('17-OCT-01','DD-MON-YY');
■ V1	NUMBER := 10;
■ V2	NUMBER := 20;
■ V3	BOOLEAN := (v1>v2);

Variables et types(6)

15

- Type dynamique
 - ▣ Déclarer une variable selon le type :
 - d'une variable déjà déclarée
 - Le type d'une colonne de table
 - Le type d'une ligne de table
 - ▣ Préfixe %type : Avec colonne ou variable
 - Nom emp.nomme%type
 - Nom2 nom%type
 - ▣ Préfixe %rowtype : Avec table
 - Contient toutes les colonnes (enregistrement)
 - Employe emp%rowtype

Variables et types(7)

16

□ Type enregistrement : (struct en C)

▣ Définition

```
Type nomStructure is record (  
    champ1 type1,  
    champ2 type2, ...);
```

▣ Exemple

```
TYPE emp2 IS RECORD (  
    matr integer,  
    nom varchar(30));  
  
employe emp2;  
employe.matr := 500;
```

Variables et types(8)

17

□ Sous types

- ▣ Définition d'un nouveau nom de type à partir des types prédéfini avec subtype (typedef en C)
- ▣ SUBTYPE sous_type IS type[(précision)][NOT NULL]
- ▣ Exemple :
Subtype titre is varchar2(15);
Titre_chapitre titre;
Titre_section titre;

Variables et types(9)

18

□ Affectation

- ▣ Plusieurs façons de donner une valeur à une variable :
 - :=
 - par la directive INTO de la requête SELECT
- ▣ Exemples :
 - `nome := 'DUPONT';`
 - `select nome INTO nom
from emp
where matr = 509;`

Variables et types(10)

19

□ Exemple

```
DECLARE
  rec emp%ROWTYPE;
  infos VARCHAR2(64);
  revenu emp.sal%TYPE ; -- rec.sal%TYPE;
BEGIN
  SELECT * INTO rec FROM emp
  WHERE ROWNUM = 1;
  revenu:=rec. sal*12;
  infos:=rec.nome || '-' || revenu || '-' ||
  TO_CHAR(rec. dateEmb,'DD/MM/YYYY');
  DBMS_OUTPUT.PUT_LINE(infos);--afficher un texte
END;
```

Opérateurs

20

□ Arithmétiques :

- ▣ *, /, +, -, **

□ Relationnels

- ▣ =, !=, <, <=, >, >=, <>

□ Logiques

- ▣ AND, OR, NOT

□ Affectation :=

□ Concaténation ||

Structures de contrôle(1)

21

□ Structures conditionnelles

▣ Choix simple

```
IF condition THEN
    instructions;
END IF;
```

▣ 2 alternatives

```
IF condition THEN
    instructions1;
ELSE
    instructions2;
END IF;
```

Structures de contrôle(2)

22

□ Structures conditionnelles

▣ Plusieurs alternatives

```
IF condition1 THEN
    instructions1;
ELSEIF condition2 THEN
    instructions2;
ELSEIF ...
    ...
ELSE
    instructionsN;
END IF;
```

Structures de contrôle(3)

23

□ Exemple

```
DECLARE
    v_sal emp.sal%TYPE;
BEGIN
    SELECT sal INTO v_sal
    FROM emp WHERE matr = 7876;
    IF v_sal BETWEEN 3000 AND 5000 THEN
        DBMS_OUTPUT.PUT_LINE('gros salaire');
    ELSIF v_sal BETWEEN 1500 AND 2999 THEN
        DBMS_OUTPUT.PUT_LINE('moyen salaire');
    ELSE
        DBMS_OUTPUT.PUT_LINE('petit salaire');
    END IF;
END;
```

Structures de contrôle(4)

24

□ Structures conditionnelles

▣ Choix

```
CASE expression
    WHEN expr1 THEN instructions1;
    WHEN expr2 THEN instructions2;
    ...
    ELSE instructionsN;
END CASE;
```

■ Expression ne peut être RECORD

Structures de contrôle(5)

25

□ Exemple

```
declare
  v char(1);
begin
  select to_char(fac_date,'D') into v from t_facture where fac_id=1;
  case v
    when '1' then dbms_output.put_line('lundi');
    when '2' then dbms_output.put_line('mardi');
    when '3' then dbms_output.put_line('mercredi');
    when '4' then dbms_output.put_line('jeudi');
    when '5' then dbms_output.put_line('vendredi');
    when '6' then dbms_output.put_line('samedi');
    when '7' then dbms_output.put_line('dimanche');
    else dbms_output.put_line('non');
  end case;
end;
```

Structures de contrôle(6)

26

□ Structures répétitives

▣ Tant que

WHILE condition LOOP

instructions;

END LOOP;

▣ Pour

FOR compteur IN [REVERSE] inf..sup LOOP

instructions;

END LOOP;

Structures de contrôle(7)

27

□ Structures répétitives

▣ Boucle générale

LOOP

instructions;

EXIT [WHEN condition];

instructions;

END LOOP;

▣ Application :

- Afficher les nombres premiers <100

Structures de contrôle(8)

28

□ Structures répétitives

▣ Boucles imbriquées et labels

- Précéder la boucle par un label entre <<label>>

- Pour sortir d'une boucle : EXIT label_boucle

- Exemple

```
BEGIN
  <<externe>>
  FOR i IN 1..2 LOOP
    <<interne>>
    FOR j IN 1..4 LOOP
      DBMS_OUTPUT.PUT_LINE('compteur externe ' || i || ' compteur
                             interne ' || j);
    END LOOP interne;
  END LOOP externe;
END;
```

Structures de contrôle(9)

29

□ Structures répétitives

▣ Exemple

compteur externe 1 compteur interne 1
compteur externe 1 compteur interne 2
compteur externe 1 compteur interne 3

Interactions avec la BD(1)

30

□ Extraire des données

- ▣ `select expr1, expr2,... into var1, var2,...`
- ▣ met des valeurs de la BD dans une ou plusieurs variables
- ▣ Le select ne doit renvoyer qu'une seule ligne
- ▣ Avec Oracle il n'est pas possible d'inclure un select sans « into » dans une procédure
- ▣ Si le select renvoie plus d'une ligne, une exception «TOO_MANY_ROWS » (ORA-01422)
- ▣ Si le select ne renvoie aucune ligne, une exception «NO_DATA_FOUND » (ORA-01403) est levée

Interactions avec la BD(2)

31

□ Modification des données

- ▣ Les requêtes SQL (`insert`, `update`, `delete`,...) peuvent utiliser les variables PL/SQL
- ▣ Les `commit` et `rollback` doivent être explicites ; aucun n'est effectué automatiquement à la sortie d'un bloc

Interactions avec la BD(3)

32

□ Modification des données

▣ Exemple :

```
declare
    v_emp emp%rowtype;
begin
    select * into v_emp from emp
    where nome = 'LEROY';
    v_emp.matr := v_emp.matr + 5;
    v_emp.nome := 'Toto';
    insert into emp
    values v_emp;
end;
```


Types de curseurs

- Toutes les requêtes SQL sont associées à un curseur
- Ce curseur représente la zone mémoire utilisée pour parser et exécuter la requête
- Le curseur peut être implicite (pas déclaré par le programmeur) ou explicite
- Les curseurs explicites servent à retourner plusieurs lignes avec un select

Attributs des curseurs

- Tous les curseurs ont des attributs que l'utilisateur peut utiliser :
 - ▣ %ROWCOUNT: nombre de lignes traitées par le curseur
 - ▣ %FOUND: vrai si au moins une ligne a été traitée par la requête ou le dernier fetch
 - ▣ %NOTFOUND: vrai si aucune ligne n'a été traitée par la requête ou le dernier fetch
 - ▣ %ISOPEN: vrai si le curseur est ouvert (utile seulement pour les curseurs explicites)

Curseurs implicites

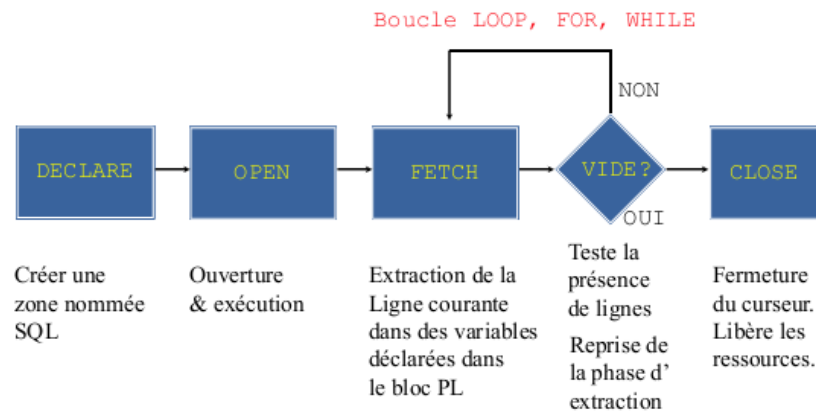
- Les curseurs implicites sont tous nommés SQL
- Exemple

```
DECLARE
    nb_lignes integer;
BEGIN
    delete from emp
    where dept = 10;
    nb_lignes := SQL%ROWCOUNT;
    ...
```

Curseurs explicites(1)

37

□ Étapes d'utilisation d'un curseur explicite



Curseurs explicites(2)

38

□ Déclaration

CURSOR nom_curseur IS ordre_select;

■ Exemple :

DECLARE

CURSOR salaires IS

select sal from emp

where dept = 10;

salaire numeric(8, 2);

total numeric(10, 2) := 0;

Curseurs explicites(3)

39

□ Ouverture du curseur

OPEN nom_curseur;

- Exécuter l'ordre select et déterminer le curseur actif

□ Extraction des données du curseur

FETCH nom_curseur **INTO** [variable1, variable2,...]
| nom_enregistrement;

- Récupérer la ligne courante du résultat du SELECT
- Par indentation, la position de chaque variable doit correspondre exactement à celle de la colonne correspondante

Curseurs explicites(4)

40

□ Fermeture du curseur

■ **CLOSE** nom_curseur;

■ Libérer l'espace mémoire

■ Rouvrir en cas de besoin

Curseurs explicites(5)

41

```
□ Exemple
DECLARE
  CURSOR salaires IS
    select sal from emp
      where dept = 10;
  salaire numeric(8, 2);
  total numeric(10, 2) := 0;
BEGIN
  open salaires;
  loop
    fetch salaires into salaire;
    exit when salaires%notfound;
    if salaire is not null then
      total := total + salaire;
      DBMS_OUTPUT.put_line(total);
    end if;
  end loop;
  close salaires; -- Ne pas oublier
  DBMS_OUTPUT.put_line(total);
END;
```

Curseurs explicites(6)

42

```
□ %rowtype et curseur
declare
  cursor c is
    select matr, nome, sal from emp;
  employe c%ROWTYPE;
begin
  open c;
  fetch c into employe;
```

Curseurs explicites(7)

43

- Boucle For pour curseurs
 - ▣ Simplifie l'usage d'un curseur : Ouverture, extraction et fermeture implicite du curseur
 - ▣ Le RECORD est également déclaré (sa portée est limitée à la boucle)
FOR nom_enregistrement IN nom_curseur LOOP
 instructions
END LOOP;

Curseurs explicites(8)

44

- Boucle For pour curseurs
 - ▣ Exemple
declare
 cursor c is
 select dept, nome from emp
 where dept = 10;
begin
 FOR employe IN c LOOP
 dbms_output.put_line(employe.nome);
 END LOOP;
end;

Curseurs explicites(9)

45

□ Boucle For pour curseurs

▣ Sans déclarer le curseur

▣ Exemple

```
begin
  FOR employe IN
    (select dept, nome from emp where dept = 10) LOOP
    dbms_output.put_line(employe.nome);
  END LOOP;
end;
```

Curseurs paramétrés(1)

46

□ Un curseur paramétré peut servir plusieurs fois avec des valeurs des paramètres différentes

□ On doit fermer le curseur entre chaque utilisation de paramètres différents (sauf si on utilise « for » qui ferme automatiquement le curseur)

□ Syntaxe

▣ **CURSOR** nom_curseur [(nom_param1 type, nom_param2 type,...)] IS ordre_select;

Curseurs paramétrés(2)

47

□ Exemple1

```
declare
  cursor c(p_dept integer) is
  select dept, nome from emp
  where dept = p_dept;
begin
  for employe in c(10) loop
    dbms_output.put_line(employe.nome);
  end loop;
  for employe in c(20) loop
    dbms_output.put_line(employe.nome);
  end loop;
end;
```

Curseurs paramétrés(3)

48

□ Exemple2

```
DECLARE
  CURSOR c1 (v_dep NUMBER) IS
  SELECT * FROM emp
  WHERE dept = v_dept;
BEGIN
  FOR i IN (SELECT * FROM emp) LOOP
    dbms_output.put_line(i.nomD);
    FOR j IN c1(i.dept) LOOP
      dbms_output.put_line(j.nome);
    END LOOP;
  END LOOP;
END;
```

Curseurs et LMD(1)

49

- Utilisation des curseurs pour supprimer ou modifier la ligne courante du curseur

- ▣ Clause FOR UPDATE

- Verrouillage des lignes pour modification ou suppression

- CURSOR IS SELECT

- FOR UPDATE [OF nom_colonne] [NOWAIT]

- ▣ Clause WHERE CURRENT OF :

- Référence à la ligne courante d'un curseur utilisée avec

- DELETE ou UPDATE

- WHERE CURRENT OF nom_curseur

Curseurs et LMD(2)

50

- Exemple

```
DECLARE
```

```
CURSOR c_sal IS
```

```
SELECT sal FROM emp
```

```
WHERE dept = 30
```

```
FOR UPDATE NOWAIT;
```

```
BEGIN
```

```
FOR v_sal IN c_sal LOOP
```

```
UPDATE emp
```

```
SET sal = v_sal.sal * 1.10
```

```
WHERE CURRENT OF c_sal;
```

```
END LOOP;
```

```
COMMIT;
```

```
END ;
```

3

Procédures et Fonctions

Définition –Sous programme

52

- Bloc PL/SQL nommé qui peut avoir des paramètres et peut être appelé
- 2 types
 - ▣ Procédure : effectue une action
 - ▣ Fonction : calcule une valeur
- Utilité
 - ▣ Enregistrer des programme dans le noyau d'Oracle
 - ▣ Stocké sous forme de pseudo-code → pas de recompilation
 - ▣ Code réutilisé par d'autres utilisateurs

Procédure –Déclaration

53

```
CREATE [OR REPLACE] PROCEDURE nomProcedure
[(liste paramètres formels)] AS | IS
    [partie déclaration]
BEGIN
    ...
[EXCEPTION
    ...]
END [nomProcedure];
```

Procédure –Liste des paramètres

54

- Déclaration d'un paramètre
 - nomParamètre [IN|OUT [NOCOPY]]|IN OUT [NOCOPY]
 - typeParamètre [:= | DEFAULT expression]
- ▣ typeParamètre: un type PL/SQL
- ▣ IN : paramètre en entrée, non modifié par la procédure
- ▣ OUT: paramètre en sortie, peut être modifié par la procédure, transmis au programme appelant
- ▣ IN OUT : à la fois en entrée et en sortie
- ▣ par défaut : IN
- ▣ NOCOPY: pour passer des références et non des valeurs

Procédure –Exemple

55

```
create or replace procedure augmentation (unDept in
integer, pourcentage in number, cout out number) is
begin
    select sum(sal) * pourcentage / 100
    into cout
    from emp
    where dept = unDept;
    update emp
    set sal = sal * (1 + pourcentage / 100)
    where dept = unDept;
end;
```

Fonction –Déclaration

56

```
CREATE [OR REPLACE] FUNCTION nomFonction
[(liste paramètres formels)] RETURN typeRetour AS | IS
    [partie déclaration]
BEGIN
    ...
    RETURN valRetour;
    ...
[EXCEPTION...]
END [nomFonction];
```

Fonction –Déclaration

57

- Liste des paramètres : idem procédure mais le mode IN est préférable
- typeRetour : type PL/SQL de valRetour

Fonction –Exemple

58

```
create or replace function euro_conv(somme in
number) return number is
    taux constant number := 0.5;
begin
    return somme * taux;
end;
```

Appel de Procédure et Fonction(1)

59

- nomProcédure [(paramètres effectifs)];
- Variable:=nomFonction [(paramètres effectifs)];
- Une fonction peut être utilisée dans du code SQL ou aussi dans des expressions PL/SQL
- Exemples :
 - Appel de procédure

```
declare
    c float;
begin
    augmentation(10, 5, c);
    dbms_output.put_line('cout = ' || c);
end;
```

Appel de Procédure et Fonction(2)

60

- Exemples :
 - Appel de fonction

```
select nome, sal, euro_conv(sal)
from emp
```

Suppression de Procédure et Fonction

61

- DROP PROCEDURE | FUNCTION nom
- Exemple :
Drop function euro_conv

4

Les Exceptions

Définition et types

63

- Un exception est une erreur ou un avertissement, prédéfinie par Oracle ou définie par le programmeur.
- 4 catégories d'exceptions :
 - nommées prédéfinies par Oracle : Oracle les déclenche, l'utilisateur peut les récupérer (WHEN nomException)
 - nommées définies par l'utilisateur : l'utilisateur les déclare, les lève dans un programme PL/SQL, et peut les récupérer (WHEN nom...)
 - anonymes prédéfinies par Oracle : Oracle les déclenche sans les nommer, l'utilisateur peut quand même les récupérer (WHEN OTHERS ou PRAGMA INIT_EXCEPTION)
 - anonymes définies par l'utilisateur : idem, mais c'est l'utilisateur qui les lève avec RAISE_APPLICATION_ERROR

Attraper une exception

64

□ Syntaxe

■ EXCEPTION

```
WHEN exception1 [OR exception2 . . .] THEN
    Traitement exception
[WHEN exception3 [OR exception4 . . .] THEN
    Traitement exception
[WHEN OTHERS THEN
    Traitement exception
```

- Un seul bloc when exécuté
- When others est le dernier

Exceptions Système(1)

65

□ Exceptions prédéfinies

Nom de l'exception	Erreur Oracle
CURSOR_ALREADY_OPEN	ORA-06511
DUP_VAL_ON_INDEX	ORA-00001
INVALID_CURSOR	ORA-01001
INVALID_NUMBER	ORA-01722
LOGIN_DENIED	ORA-01017
NO_DATA_FOUND	ORA-01403
NOT_LOGGED_ON	ORA-01012
PROGRAM_ERROR	ORA-06501
STORAGE_ERROR	ORA-06500
TIMEOUT_ON_RESOURCE	ORA-00051
TOO_MANY_ROWS	ORA-01422
TRANSACTION_BACKED_OUT	ORA-00061
VALUE_ERROR	ORA-06502
ZERO_DIVIDE	ORA-01476

Exceptions Système(2)

66

□ Exemple

When NO_DATA_FOUND Then

Dbms_output.put_line('Résultat vide : ' || SQLERRM);

■ SQLERRM : fonction retournant le message d'erreur associé à l'exception levée

□ Exceptions non-prédéfinies

■ Identifiées par un numéro

■ Utilisation de PRAGMA EXCEPTION_INIT

Exceptions Système(3)

67

□ Exemple

■ Exception numéro -2292 : violation contrainte d'intégrité

CREATE OR REPLACE PROCEDURE supprimer_dep (v_deptno IN dept.deptno%type) AS

e_dep_nonvide EXCEPTION;

PRAGMA EXCEPTION_INIT (e_dep_nonvide, -2292) ;

BEGIN

DELETE FROM dept WHERE deptno = v_deptno;

COMMIT;

EXCEPTION

WHEN e_dep_nonvide THEN

DBMS_OUTPUT.PUT_LINE ('Suppression impossible du département ' || TO_CHAR(v_deptno) || '. Des employés y existent. ');

END;

Exceptions Développeur(1)

68

□ Étapes

■ Nommer l'exception : la déclarer de type EXCEPTION

■ Lever l'exception avec RAISE

■ Attraper l'exception comme une exception prédéfinies nommée

Exceptions Développeur(2)

69

□ Exemple

```
CREATE PROCEDURE modif_produit(nump IN produit.prodid%type, desc IN
produit.descrip%type )
e_produit_invalide EXCEPTION;
BEGIN
    UPDATE produit
    SET descrip = desc
    WHERE prodid = nump;
    IF SQL%NOTFOUND THEN
        RAISE e_produit_invalide;
    END IF;
    COMMIT;
EXCEPTION
    WHEN e_produit_invalide THEN
        DBMS_OUTPUT.PUT_LINE( 'Numéro de produit incorrect. ');
END ;
```

Exceptions Développeur(3)

70

□ Fonction pour les exceptions

- SQLCODE : retourne le numéro de l'exception
- SQLERRM : retourne le message d'erreur associé

□ Exceptions utilisateur anonymes

RAISE_APPLICATION_ERROR(numéro, message
[, TRUE | FALSE]);

- Numéro : entre -20000 et -20999
- TRUE : placer sur la pile des erreurs
- FALSE : remplace les exceptions précédentes

Propagation d'une Exception

71

□ Une exception est déclenchée :

- PL/SQL cherche un gestionnaire dans la partie Exception du bloc courant (WHEN adéquat)
- Si pas de gestionnaire : on cherche dans le bloc englobant...
- Si aucun : PL/SQL envoie un message d'exception non gérée à l'application appelante

5

Les Déclencheurs

Définition

73

- Déclencheur ou trigger
 - ▣ Un bloc de code PL/SQL associé à une table, vue, schéma ou base de données
 - ▣ Exécuté suite à un événement et ne peut être appelé explicitement
- Types
 - ▣ Trigger d'application
 - Événement dans une application
 - ▣ Trigger de base de données
 - Suite à une opération sur les données (LMD) ou événement système (connexion, déconnexion, ...) sur un schéma de BD

Création de trigger

74

```
CREATE TRIGGER nom
    (événement déclenchant le trigger)
    ...
    (contrainte optionnelle du trigger)
BEGIN
    (action du trigger)
END;
```

Trigger de LMD(1)

75

```
CREATE TRIGGER nom
    BEFORE | AFTER INSERT | UPDATE | DELETE [OF
    colonne] ON table
    [FOR EACH ROW]
    WHEN (condition)
BEGIN
    (action du trigger)
END;
```

Trigger de LMD(2)

76

```
□ Exemple
CREATE OR REPLACE TRIGGER secu_emp
    BEFORE INSERT ON emp
BEGIN
    If(to_char(sysdate,'day') in ('samedi','dimanche') or
    to_char(sysdate,'HH24') not between '08' and '18H') then
        raise_application_error(-20500,'Vous ne pouvez insérer
        dans la table emp que pendant les heures de bureaux!');
    End if;
END;
```

Trigger de LMD(2)

77

□ Utilisation des prédicats conditionnels

```
CREATE OR REPLACE TRIGGER secu_emp
BEFORE INSERT OR UPDATE OR DELETE ON emp
BEGIN
    If(to_char(sysdate,'day') in ('samedi','dimanche') or to_char(sysdate, 'HH24') not
    between '08' and '18H') then
        If INSERTING then
            raise_application_error(-20500,
                'Vous ne pouvez insérer dans la table emp que pendant les heures de bureaux!');
        elsif DELETING then
            raise_application_error(-20501,
                'Vous ne pouvez supprimer de la table emp que pendant les heures de bureaux!');
        elsif UPDATING then
            raise_application_error(-20502,
                'Vous ne pouvez modifier la table emp que pendant les heures de bureaux!');
        else
            raise_application_error(-20503,
                'Vous ne pouvez insérer dans la table emp que pendant les heures de bureaux!');
        End if;
    End if;
END;
```

:New et :Old(1)

78

- Référence au nouvel enregistrement et l'ancien enregistrement
- Selon le type de l'opération
 - ▣ INSERT
 - :New seulement est accessible
 - Modifiable si BEFORE
 - ▣ UPDATE
 - :New et :Old sont accessibles
 - :New est modifiable si BEFORE
 - ▣ DELETE
 - :Old seulement est accessible

:New et :Old(2)

79

□ Exemple :

- CREATE OR REPLACE TRIGGER verifiersalaire
 - BEFORE UPDATE ON emp
 - FOR EACH ROW
- BEGIN
 - IF not (:New.poste='Directeur' AND :New.sal>2000) then
 - Raise_application_error(-20300,'Salaire incorrect');
 - END IF;
- END ;

Manipulation des triggers

80

- Activer/désactiver
 - ▣ ALTER TRIGGER nom ENABLE | DISABLE;
- Activer/désactiver les triggers d'une table
 - ▣ ALTER TABLE nomTable ENABLE | DISABLE ALL TRIGGERS;
- Supprimer
 - ▣ DROP TRIGGER nom;

Triggers de base de données

81

□ Événement Utilisateur (USER)

- ▣ Connexion : LOGON
- ▣ Déconnexion : LOGOFF

□ Exemple :

```
CREATE OR REPLACE TRIGGER log_connexion
  AFTER LOGON ON SCHEMA
  BEGIN
    INSERT INTO journal(user_id,date_action,action) VALUES (USER,
    SYSDATE,'Connexion');
  END;
```